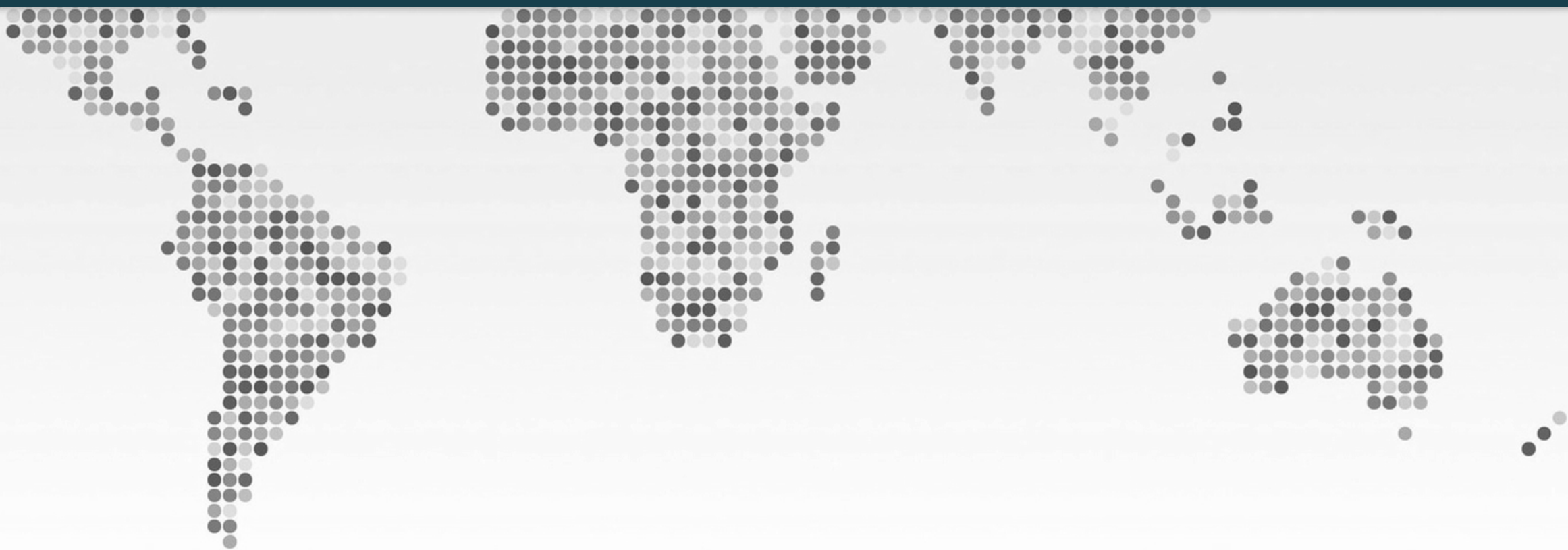


Software Engineering for Artificial Intelligence

Technical Debt in the GenAI Era



Software Engineering for Artificial Intelligence: Technical Debt in the GenAI Era

Technical Debt in the GenAI Era: Prologue

In the previous lectures, we explored multiple aspects connected to the quality assurance of AI-enabled systems, finding that AI systems already accumulate data debt, model debt, or even system-level debt like Pipeline Jungle and similar.

Furthermore, we explored the concept of Agentic AI as a composition of software systems, not just models.

This lecture combines the two, elaborating how technical debt may emerge in GenAI and GenAI Agents. NB: Most of the arguments in the lecture are emerging, so there is no ground truth here, just reasonable consideration!

Working Hypothesis. In GenAI systems, prompts become *executable specifications*. In other terms, prompts are not documentation, but code-like artifacts.

The rationale behind this working hypothesis is that prompts:

- 1 Act as *architectural connectors* between models, tools, and goals;
- 2 Define system-level behavior *without being compiled or statically verified*;
- 3 Replace traditional control structures with *natural-language orchestration logic*;
- 4 Are embedded in agentic loops, where small changes can have non-local effects.

Technical Debt in the GenAI Era: Prompts as Code

To better understand the code-like nature of prompts, let's better understand the context we are talking about, namely that of a GenAI-enabled system.

Indeed, just to clarify, we are not considering prompting as an ad-hoc interaction between a user and an LLM, i.e., what we do when chatting with ChatGPT.

Instead, we adopt the perspective of a GenAI-enabled software system, where:

- 1 Prompts are *embedded in the system*, not written by end users;
- 2 They are invoked *automatically and repeatedly*, often at scale;
- 3 They are expected to produce *consistent, reliable, and predictable behavior across executions*;
- 4 Their outputs *directly affect* downstream components, decisions, or actions.

In this setting, prompts cannot be treated as disposable instructions; must be designed, maintained, and evolved like other software artifacts; and become a potential source of system-level technical debt if poorly managed.

So, if we think at prompts like executable specifications embedded in software systems, then they can degrade over time. This degradation may manifest as *prompt-level technical debt*.

Software Engineering for Artificial Intelligence: Technical Debt in the GenAI Era

Technical Debt in the GenAI Era: Prompts as Code

In addition, when moving from standalone LLM components to GenAI agents, prompts no longer operate in isolation.

A GenAI agent is a composed software system that pursues explicit goals, interacts with a mediated environment, acts through delegated tools, and operates under uncertainty.

Within this architecture, prompts play a *structural* role:

- 1 They *instantiate the agent's goals* at runtime;
- 2 They *mediate perception*, shaping how observations are interpreted;
- 3 They *constrain and enable actions*, determining how tools are invoked;
- 4 They *coordinate reasoning* across agent loops, memory, and planning steps.

As a consequence, prompts are not peripheral inputs: they are architectural artifacts that bind together the agent's components.

If we think of the agent as an architecture composed of perception, reasoning, memory, and action, then prompts are the mechanism that *configures how these components collaborate at runtime*.

So, they are architectural control artifacts that shape system behavior, which give them much more importance than simple textual artifacts.

Software Engineering for Artificial Intelligence: Technical Debt in the GenAI Era

Technical Debt in the GenAI Era: Prompt Debt

Prompt Debt. It arises when prompt design decisions are deliberately optimized for short-term effectiveness or delivery speed, at the expense of long-term qualities such as maintainability, robustness, evolvability, and architectural clarity.

Example. A company is developing a GenAI-based customer support agent that classifies incoming tickets, decides whether to escalate them, and optionally triggers follow-up actions via tools.

According to the requirements of the agent, it has to behave conservatively (hence avoiding false escalations) and produce outputs that downstream services can parse.

You are a customer support agent.

Classify the ticket as LOW, MEDIUM, or HIGH urgency.

Escalate ONLY if the issue is severe, recurring, and blocking business operations.

Be conservative: when in doubt, choose a lower urgency.

Output ONLY a JSON object with fields {urgency, escalate}.

Do NOT include explanations or additional text.

The team decides to encode classification logic, escalation policy, and output constraints in the prompt, relying on the LLM to follow them. This choice is made because the model already “understands” natural language policies; in addition, prompt changes are fast and cheap.

Software Engineering for Artificial Intelligence: Technical Debt in the GenAI Era

Technical Debt in the GenAI Era: Prompt Debt

Prompt Debt. It arises when prompt design decisions are deliberately optimized for short-term effectiveness or delivery speed, at the expense of long-term qualities such as maintainability, robustness, evolvability, and architectural clarity.

You are a customer support agent.

Classify the ticket as LOW, MEDIUM, or HIGH urgency.

Escalate ONLY if the issue is severe, recurring, and blocking business operations.

Be conservative: when in doubt, choose a lower urgency.

Output ONLY a JSON object with fields {urgency, escalate}.

Do NOT include explanations or additional text.

Why may this be considered a debt?

The team knowingly accepts that (1) business rules are *embedded in untyped natural language*; (2) *policy changes require prompt edits*, not configuration changes; (3) *behavior is entangled with model interpretation*; (4) testing and validation are limited to *end-to-end behavior*. In other terms, **prompt text has replaced explicit architectural control**.

This has consequences. Indeed, policy changes require careful prompt rewriting, small prompt edits have non-local effects on agent behavior, the prompt becomes harder to understand and reason about, and migrating to another model becomes risky.

Software Engineering for Artificial Intelligence: Technical Debt in the GenAI Era

Technical Debt in the GenAI Era: Prompt Debt

Prompt Debt. It arises when prompt design decisions are deliberately optimized for short-term effectiveness or delivery speed, at the expense of long-term qualities such as maintainability, robustness, evolvability, and architectural clarity.

So, the system may possibly behave correctly today, but its design knowledge lives in the prompt, not in the architecture.

An architecturally sound solution would have been, instead:

Pre-processor

This normalizes incoming tickets and extracts structured signals, e.g., metadata and customer, context before the data is passed to the LLM.

LLM classifier

This is responsible for classifying the ticket and producing an output that includes the predicted category, an urgency score, and supporting evidence.

Validator

This checks that the LLM output conforms to a predefined schema, satisfies content constraints, and meets consistency and confidence requirements.

Policy engine

This applies explicit business rules to decide whether the ticket should be escalated, independently of how the classification was produced.

Action orchestrator

This executes system actions based on policy, enforcing permissions, safety constraints, and operational limits on any external tools or services.

Observability layer

This continuously monitors system behavior by collecting logs and traces, tracking model drift, and supporting evaluation of the agent's decisions.

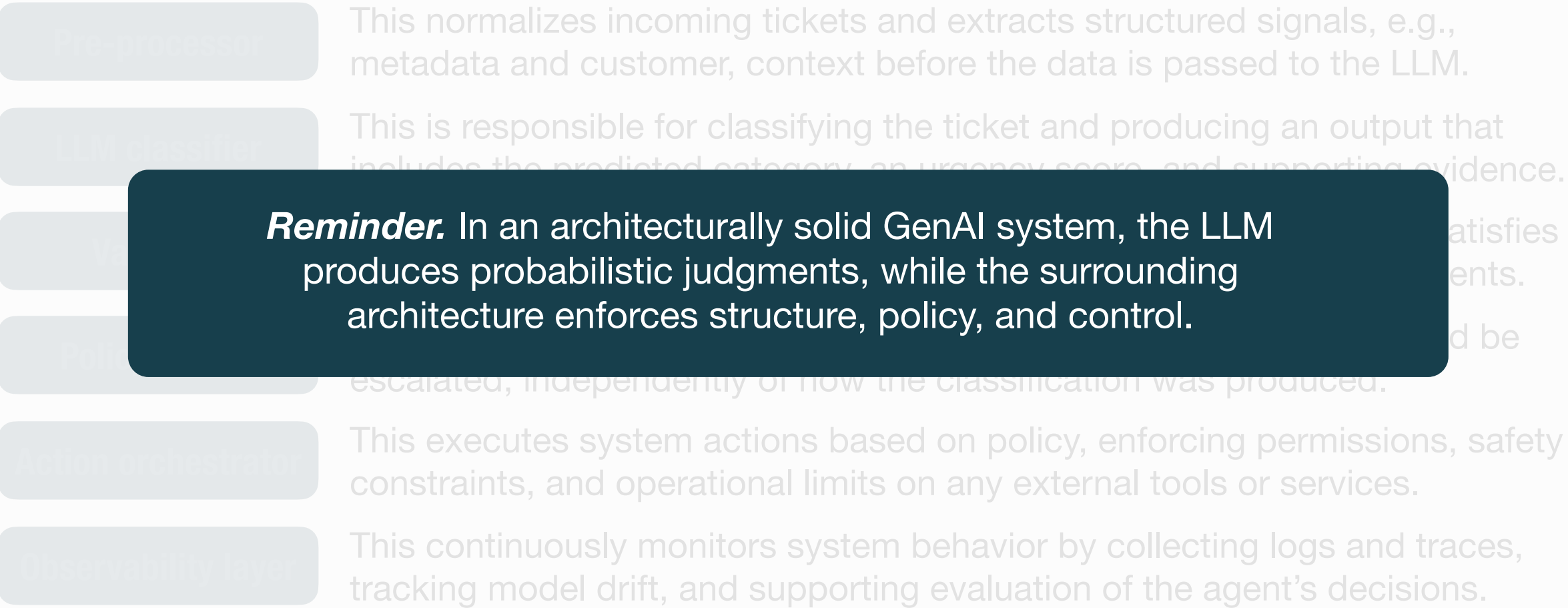
Software Engineering for Artificial Intelligence: Technical Debt in the GenAI Era

Technical Debt in the GenAI Era: Prompt Debt

Prompt Debt. It arises when prompt design decisions are deliberately optimized for short-term effectiveness or delivery speed, at the expense of long-term qualities such as maintainability, robustness, evolvability, and architectural clarity.

So, the system may possibly behave correctly today, but its design knowledge lives in the prompt, not in the architecture.

An architecturally sound solution would have been, instead:



Software Engineering for Artificial Intelligence: Technical Debt in the GenAI Era

Technical Debt in the GenAI Era: Prompt Debt

Prompt Debt. It arises when prompt design decisions are deliberately optimized for short-term effectiveness or delivery speed, at the expense of long-term qualities such as maintainability, robustness, evolvability, and architectural clarity.

Prompt Smell. It is a recurring design weakness in prompts that may not cause immediate failures, but that undermine the long-term quality of LLM-enabled systems.

As in the case of traditional code smells, they are not bugs (namely, they do cause failures), but are indicators of potential technical debt.

At the time of this lecture, we still know only a few things about prompt smells - we are indeed in the moment where we can just classify few types based on syntax or semantic.

Perhaps more importantly, most of the existing prompt smells refer to text-to-text GenAI models/systems, hence missing other types of GenAI (e.g., text-to-image, text-to-video, image-to-video). The missing ones are very likely the most problematic!

Even more importantly, prompt debt has not yet been formally defined - the definition and example discussed so far are my own interpretation, inspired by classical technical debt theory.

In this lecture, we treat prompt smells as early warning signals of prompt debt.

Software Engineering for Artificial Intelligence: Technical Debt in the GenAI Era

Technical Debt in the GenAI Era: Prompt Debt

Prompt Smell. It is a recurring design weakness in prompts that may not cause immediate failures, but that undermine the long-term quality of LLM-enabled systems.

Syntactic smells

They concern how the prompt is written, independently of its meaning or intent. Typical syntactic smells affect grammar, formatting, structure, and readability, and may hinder the model's ability to correctly parse and follow the instructions.

Semantic smells

They concern whether the prompt provides sufficient context, expresses clear objectives, aligns style and constraints with the intended output, and avoids biased or unsafe framing. Semantic smells affect how the model interprets the task and may lead to brittle, inconsistent, or misaligned behavior even when the prompt is syntactically correct.

Toward Measuring Prompt Quality: A Preliminary Investigation on Prompt Smells

Antonio Della Porta*, Gianmario Voria*, Andrea Abbate*, Raffaele Sulipano*,

Stefano Lambiase†, Gemma Catolino*, and Fabio Palomba*

*Software Engineering (SeSa) Lab, University of Salerno, Italy

†Aalborg University, Denmark

Abstract—Large Language Models (LLMs) are increasingly embedded in software engineering workflows, yet the quality of the prompts used to guide these models remains largely unassessed. Poorly formulated prompts can lead to ambiguity, degraded output quality, or biased and unsafe model behavior. In this paper, we conduct a preliminary exploratory study aimed at establishing an initial foundation for systematic prompt-quality evaluation. First, we perform a rapid review of the literature on prompt engineering and analyze real-world prompts to derive a preliminary taxonomy of *prompt smells*—recurring patterns that signal potential quality issues. Each smell is operationalized through a quantitative metric capturing aspects such as grammatical correctness, readability, contextual adequacy, complexity-length, formality mismatch, and bias.

We then apply these metrics to an existing corpus of developer-written prompts and analyze their relationships using correlation analysis and multiple regression models. Our results show that syntactic metrics (e.g., grammar, readability) are strongly interrelated and largely explain the composite Prompt Quality Score, whereas higher-level smells such as contextual relevance, formality mismatch, and bias exhibit weak correlations and low explained variance. These findings suggest that many important prompt-quality issues operate independently of surface-level text properties and may require more semantically informed assessment techniques. Overall, this study offers an initial conceptual and empirical basis for the systematic evaluation of prompt quality, paving the way for more robust prompt engineering practices and automated quality-checking tools.

Index Terms—Prompt Quality; Prompt Smells

I. INTRODUCTION

LLMs are nowadays widely adopted to support natural language and software engineering (SE) tasks, mostly without task-specific fine-tuning [1]. In this setting, *prompts*, i.e., the natural language instructions provided as a query to the model to guide it towards a precise answer [1], are the main artifact used by practitioners to control models' responses: instead of changing model parameters, users write prompts to instruct the model on what to do, how to do it, and in which style [2]. This has led the SE community to introduce a new discipline, i.e., *prompt engineering*, that investigates specifically *how* to engineer prompts to guide LLMs in reliably providing qualitative answers. Recent surveys list many prompt engineering strategies, and show that the way a task is written in the prompt can be as important as the choice of model itself [3].

Although prompt engineering is now common, most academic work evaluates LLMs based on their task outputs, rather

than on the intrinsic **qualitative properties of the prompts as SE artifacts**. Surveys on LLM evaluation organize the literature around evaluation tasks, benchmarks, and metrics like accuracy, BLEU, ROUGE, F1, human judgments, or safety scores, and treat the prompt as part of the experimental setup rather than as an object of evaluation on its own [4], [5]. A recent review by Joshi et al. [6] reaches a similar conclusion and summarizes LLM evaluation primarily in terms of task-level and system-level metrics, while only briefly noting emerging work on prompt-level metrics. There is still no shared, multi-dimensional model of *prompt quality* that can be computed automatically and studied across datasets.

In common SE research, a related problem is addressed through the concept of *code smells* [7]. These are surface-level indicators in source code that often point to deeper design problems, such as very long methods, large classes, duplicated code, or overly long parameter lists. Over time, catalogs of code smells have been created and integrated into tools to guide refactoring and improve maintainability [8]. A code smell is not a bug; it is a warning that a piece of code might be hard to understand, modify, or test in the future. *In this paper, we position prompts as the artifact that engineers use to guide LLMs in providing reliable and high-quality responses, and investigate intrinsic qualitative aspects of such prompts.* Many low-quality prompts show obvious surface issues, for example, “*Explain this*” with no context, or “*Give me a perfect answer, you must be 100 percent correct*”, with no task description. Other prompts include unclear pronouns (e.g., “*Do it like before*”), inconsistent tone, or biased framing (e.g., “*Explain why group X is worse than group Y*”). These are not syntax errors, but they are signs that the prompt may lead to brittle, confusing, or unsafe model behavior. We refer to such recurring surface issues as **prompt smells**.

However, in the current landscape of *quality assurance for LLMs*, there is no consolidated taxonomy of prompt smells that combines insights from the literature with observations from real prompts. Furthermore, there is a lack of operational metrics that map different dimensions of these quality issues to concrete quantitative indicators, such as grammatical quality, structural complexity, contextual adequacy, style, and bias. Finally, there is very limited empirical work on how such metrics relate to one another across realistic prompt corpora, to understand what characterizes prompt quality.

Software Engineering for Artificial Intelligence: Technical Debt in the GenAI Era

Technical Debt in the GenAI Era: Prompt Debt

Prompt Smell. It is a recurring design weakness in prompts that may not cause immediate failures, but that undermine the long-term quality of LLM-enabled systems.

Syntactic smells

Grammar and Surface-Form Smell

It occurs when a prompt has grammatical errors, misspellings, or malformed sentences that reduce clarity and precision.

Analyze the log file and explain what error **are** happened and why **it cause** the failure.

smelly version



Analyze the log file and explain what errors happened and why they caused the failure.

refactored version

Why this is a smell. The task is understandable, but grammatical inaccuracies introduce ambiguity. The model may misinterpret relationships between actions, causes, and effects, leading to inconsistent explanations.

How it can be refactored. The refactoring is fairly easy in this case, as it revolves around addressing the grammatical inconsistencies of the original sentence.

Software Engineering for Artificial Intelligence: Technical Debt in the GenAI Era

Technical Debt in the GenAI Era: Prompt Debt

Prompt Smell. It is a recurring design weakness in prompts that may not cause immediate failures, but that undermine the long-term quality of LLM-enabled systems.

Syntactic smells

Formatting and Structural Smell

This smell arises when a prompt lacks a clear structure or mixes multiple instructions without explicit separation.

Read the following code explain the bug suggest a fix output in JSON do not include comments be concise.

smelly version

Why this is a smell. Multiple requirements are compressed into a single unstructured sentence. The absence of structure makes it unclear how many tasks are requested and which constraints apply to which part of the output.

Task. Analyze the following code and identify the main bug.

Output Requirements. Suggest a fix for the identified bug.

Format. Return the result as a JSON object.

Constraints. Do not include comments or explanatory text. Keep the output concise.

refactored version

Software Engineering for Artificial Intelligence: Technical Debt in the GenAI Era

Technical Debt in the GenAI Era: Prompt Debt

Prompt Smell. It is a recurring design weakness in prompts that may not cause immediate failures, but that undermine the long-term quality of LLM-enabled systems.

Semantic smells

Missing/Insufficient Context Smell

This smell occurs when a prompt does not provide enough information about the task, domain, or expected behavior.

Optimize this function

smelly version

Why this is a smell. The prompt does not specify whether optimization refers to performance, memory usage, readability, or energy efficiency. The model must guess the intent, leading to variable and unreliable results.

How it can be refactored. You refactor the prompt by making the intent explicit and by adding the minimum context needed for the system to behave consistently.

You are improving a production function.

Optimize the function for runtime performance while keeping the same observable behavior.

Return the optimized code and a short explanation of the key improvement.

refactored version

Software Engineering for Artificial Intelligence: Technical Debt in the GenAI Era

Technical Debt in the GenAI Era: Prompt Debt

Prompt Smell. It is a recurring design weakness in prompts that may not cause immediate failures, but that undermine the long-term quality of LLM-enabled systems.

Semantic smells

Over-Complexity - Excessive Length

This smell arises when a prompt includes too many constraints, exceptions, or secondary goals that dilute the main objective.

Summarize the document, but keep technical terms, simplify the language, preserve all details, reduce length by half, use bullet points, avoid bullet points when inappropriate, and adapt the tone depending on the audience.

smelly version



Task. Summarize the document for a technical audience.

Primary Objective. Reduce the length of the document by approximately 50% while preserving the technical information.

Secondary Guidelines. Use clear and accessible language, but retain essential technical terms. Prefer bullet points when they improve clarity.

Output. Summarize as a structured list of bullet points.

refactored version

Why this is a smell. The prompt contains conflicting/excessive requirements. Instead of improving control, the prompt makes the expected behavior harder to reason about and harder to evolve.

How it can be refactored. The refactoring reduces over-complexity by making priorities explicit and eliminating conflicting requirements.

Software Engineering for Artificial Intelligence: Technical Debt in the GenAI Era

Technical Debt in the GenAI Era: Prompt Debt

Prompt Smell. It is a recurring design weakness in prompts that may not cause immediate failures, but that undermine the long-term quality of LLM-enabled systems.

As some of you may have heard already, smells are typically diagnosed - and potentially automatically detected - through static analysis tools (or through advanced mechanisms, like machine learning or even Large Language Models).

Independently from the mechanism used for detection, a key enabler for **making smells observable** is represented by the ability to **associate artifacts with measurable indicators**.

Prompt Metric. A prompt metric is a quantitative indicator that captures a specific, observable property of a prompt as a software artifact.

More specifically, prompt metrics are used to approximate different dimensions of prompt quality, e.g., prompt length. Also, to support the identification and assessment of prompt smells, e.g., by defining thresholds over length to identify excessively long prompts. And, thus, to enable systematic analysis, comparison, and monitoring of prompts over time.

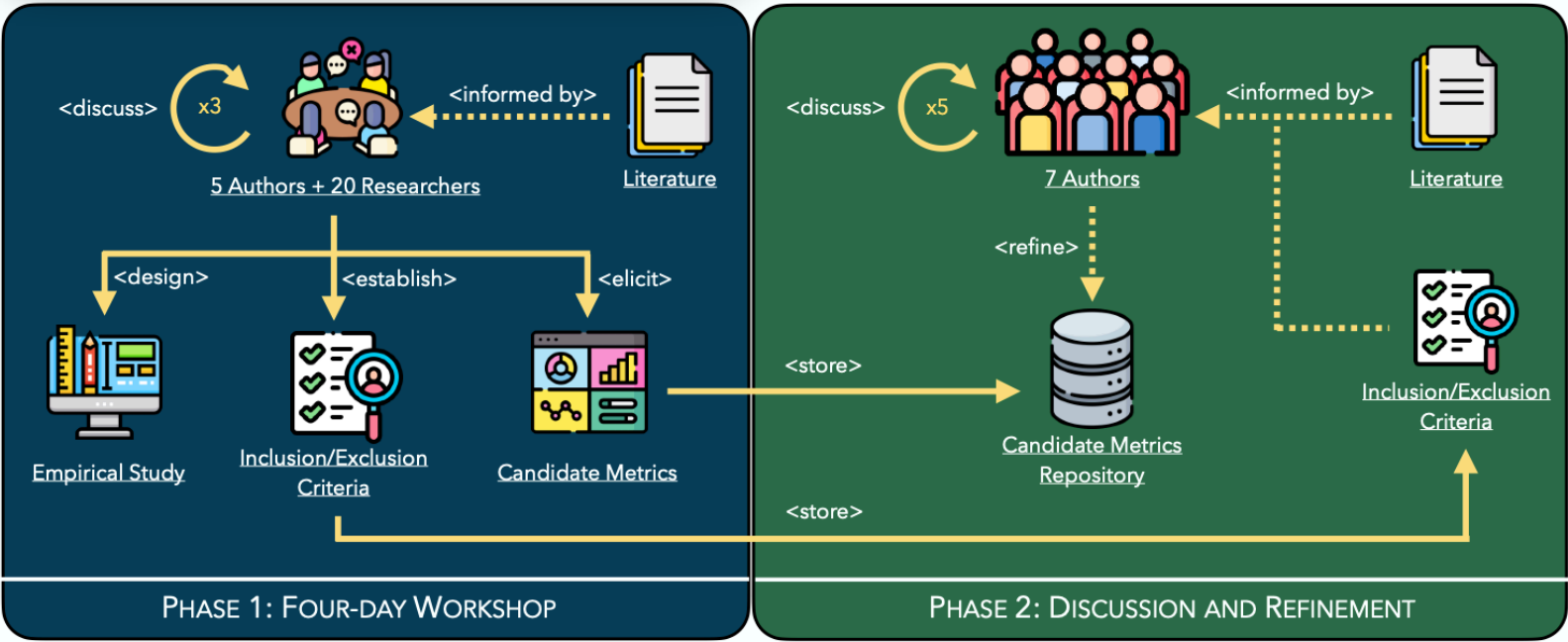
NB: A prompt metric does **not** directly measure correctness or usefulness.

Software Engineering for Artificial Intelligence: Technical Debt in the GenAI Era

Technical Debt in the GenAI Era: Prompt Debt

Prompt Metric. A prompt metric is a quantitative indicator that captures a specific, observable property of a prompt as a software artifact.

A prompt metric is a **quantitative proxy** that captures a **measurable property of a prompt** to support quality assessment and smell detection.



IEEE TRANSACTIONS ON SOFTWARE ENGINEERING

1

Lost in Translation or Clear as Day?
Understanding Prompt Quality and Its Relation
to LLM-based Code Generation

Antonio Della Porta , Stefano Lambiase , Valeria Pontillo , João F. Ferreira , Akond A. Rahman ,
Chaiyong Ragkhitwetsagul , Fabio Palomba

Abstract—Prompt engineering has emerged as a key practice to guide Large Language Models (LLMs) in code generation tasks, under the assumption that better prompts yield better outputs. Yet little is known about which characteristics of prompts actually drive variations in output quality. In this paper, we take a first step toward addressing this gap by investigating how measurable properties of prompts relate to the quality of generated code. We conducted an empirical study of real-world, single-turn interactions between developers and ChatGPT collected from GitHub. Prompts were characterized along readability and structural dimensions, while output quality was measured as semantic similarity to committed code, perceived usefulness, and correctness. Regression analysis shows that readability metrics are strong predictors of usefulness and correctness, whereas structural features such as the number of sentences and task type significantly affect similarity. Overall, our findings provide evidence that prompt quality is multi-dimensional and that measurable characteristics can serve as indicators of the quality of LLM-generated code in software engineering.

Index Terms—Prompt Quality; Prompt Engineering; Empirical Software Engineering; LLM.

1 INTRODUCTION

LARGE LANGUAGE MODELS (LLMs) are reshaping software engineering, powering tasks from code generation [1], [2] and summarization [3], [4] to code review, debugging, and testing [5]–[9]. Despite these promising advances, LLMs may pose significant threats to software quality, particularly when used for code generation [10]. Recent studies show that they can (i) hallucinate, producing code that diverges from developers’ intent [11]; (ii) inject security weaknesses into otherwise correct-looking programs [12]; and (iii) reduce code quality and maintainability [13], [14]. These issues may arise either from the stochastic nature of LLMs and their training data [15], [16], or from the formulation of the prompts, i.e., the requests provided by developers [17]. This latter has received increasing attention under the label of *prompt engineering*, i.e., the systematic study and practice of crafting prompts to guide LLMs toward more accurate and reliable outputs [18].

Much of the existing research in this field has focused on the effectiveness of prompting strategies, proposing approaches such as *prompt patterns* [19], iterative refinement

methods [20], and reasoning-oriented techniques [21]. These studies provide valuable evidence that prompt design matters, yet they typically assess quality *indirectly*, by evaluating whether a given strategy improves task outcomes. This pragmatic perspective is useful: much like software testing, it reveals whether a specific input works under particular conditions. Nonetheless, current approaches consider prompt quality only through outcomes, treating it as a *by-product of output evaluation* rather than a construct with its own measurable characteristics. As a result, the **intrinsic properties of prompts themselves** remain underexplored. This leaves open a more fundamental issue: *What, in itself, makes a prompt “good”?* Without a principled understanding of this concept, it is hard to know whether existing strategies truly optimize the aspects of a prompt that matter for model performance, to develop systematic methods for improving prompts beyond trial-and-error, or to compare alternative strategies on common grounds. As such, the lack of a principled notion of prompt “goodness” represents a key gap in current research. In this paper, we take a first step toward addressing this gap. Our conjecture is that, since the ultimate goal of a “good” prompt is to produce a “good” output, its definition should be based on the idea that any measure of prompt quality must at least correlate with variations in the final output quality of LLMs.

● Objective of the Work

Building on these considerations, this paper presents an empirical investigation aimed at identifying prompt-related metrics that correlate with variations in the quality of code generated by LLMs. Our ultimate ob-

Software Engineering for Artificial Intelligence: Technical Debt in the GenAI Era

Technical Debt in the GenAI Era: Prompt Debt

Prompt Metric. A prompt metric is a quantitative indicator that captures a specific, observable property of a prompt as a software artifact.

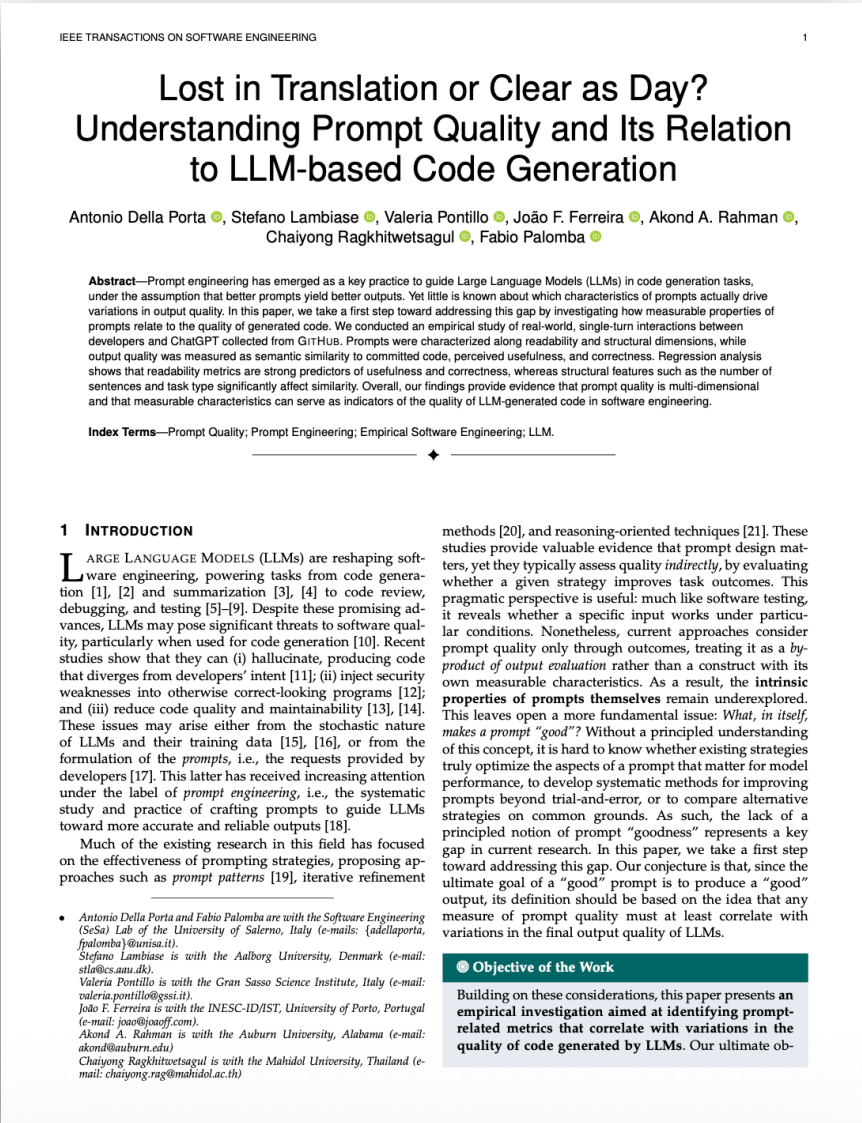
A prompt metric is a **quantitative proxy** that captures a **measurable property of a prompt** to support quality assessment and smell detection.

Readability-related

They quantify the linguistic clarity and expressiveness of a prompt. They capture how easy it is to read, parse, and understand the intent of the request, independently of the specific task being performed. The underlying assumption is that **clearer prompts reduce ambiguity**.

Structural

They capture how a prompt is organized and composed, rather than how it is phrased linguistically. These metrics reflect the idea that **how information is structured matters as much as how it is written**.



Software Engineering for Artificial Intelligence: Technical Debt in the GenAI Era

Technical Debt in the GenAI Era: Prompt Debt

Prompt Metric. A prompt metric is a quantitative indicator that captures a specific, observable property of a prompt as a software artifact.

Readability-related

Flesch Reading Ease

Flesch Reading Ease estimates how easy a text is to read based on sentence length and word complexity.

Please modify the function so that it handles null values correctly and improves performance.

metric value $\approx$ 65–70; scale: 0–100 (higher is easier).

Interpretation. This prompt is short, uses simple sentence structure, and avoids unnecessary jargon. A high readability score suggests that the intent is communicated clearly, reducing the risk of misinterpretation.

How it can be used for quality assessment. A score in this range indicates plain, accessible language that can be easily parsed by both humans and models. This suggests low linguistic ambiguity, clear expression of intent, and reduced cognitive load during interpretation.

Should the value be lower than a given threshold (e.g., <30), then this would potentially be a readability smell, hence suggesting that the prompt should be refined or refactored.

Software Engineering for Artificial Intelligence: Technical Debt in the GenAI Era

Technical Debt in the GenAI Era: Prompt Debt

Prompt Metric. A prompt metric is a quantitative indicator that captures a specific, observable property of a prompt as a software artifact.

Structural

Number of Sentences

The metric counts how many sentences compose the prompt.

The following function fails when the input list is empty. Identify the cause of the bug and modify the code so that empty inputs are handled correctly.

metric value = 2; scale: 1–N.

Interpretation. Increasing the number of sentences may correlate with better articulation of intent, explicit assumptions, and reduced reliance on implicit inference. At the same time, an excessive number of sentences may indicate over-complexity, entangled requirements, or competing objectives.

How it can be used for quality assessment. The number of sentences can be used as a coarse indicator of prompt completeness and structural clarity. A very low number of sentences may indicate that the prompt is underspecified, leading to a *Missing Context* smell. A high number of sentences may indicate that the prompt encodes multiple concerns, leading to *Over-Complexity* and *Excessive Length* smells.

Software Engineering for Artificial Intelligence: Technical Debt in the GenAI Era

Technical Debt in the GenAI Era: Prompt Quality Assurance Challenges

To sum up, the current state of the practice proposes multiple early forms of quality assurance methods for prompt quality and analysis:

Static prompt metrics

These metrics make prompts *observable*, but they do not guarantee semantic or behavioral quality.

Prompt engineering guidelines and best practices

Many practical recommendations exist on how to write “good prompts”. However, these guidelines are often informal, tool-specific, and rarely connected to maintainability or technical debt concerns.

End-to-end output-based evaluation

Prompt quality is assessed indirectly by evaluating the correctness of the generated output. This approach does not support diagnosing prompt design issues, nor does it help anticipate long-term evolution problems.

Dynamic prompt analysis

There are only few systematic approaches to analyze how a prompt behaves under variations of input, context, or model configuration.

Structured prompt testing

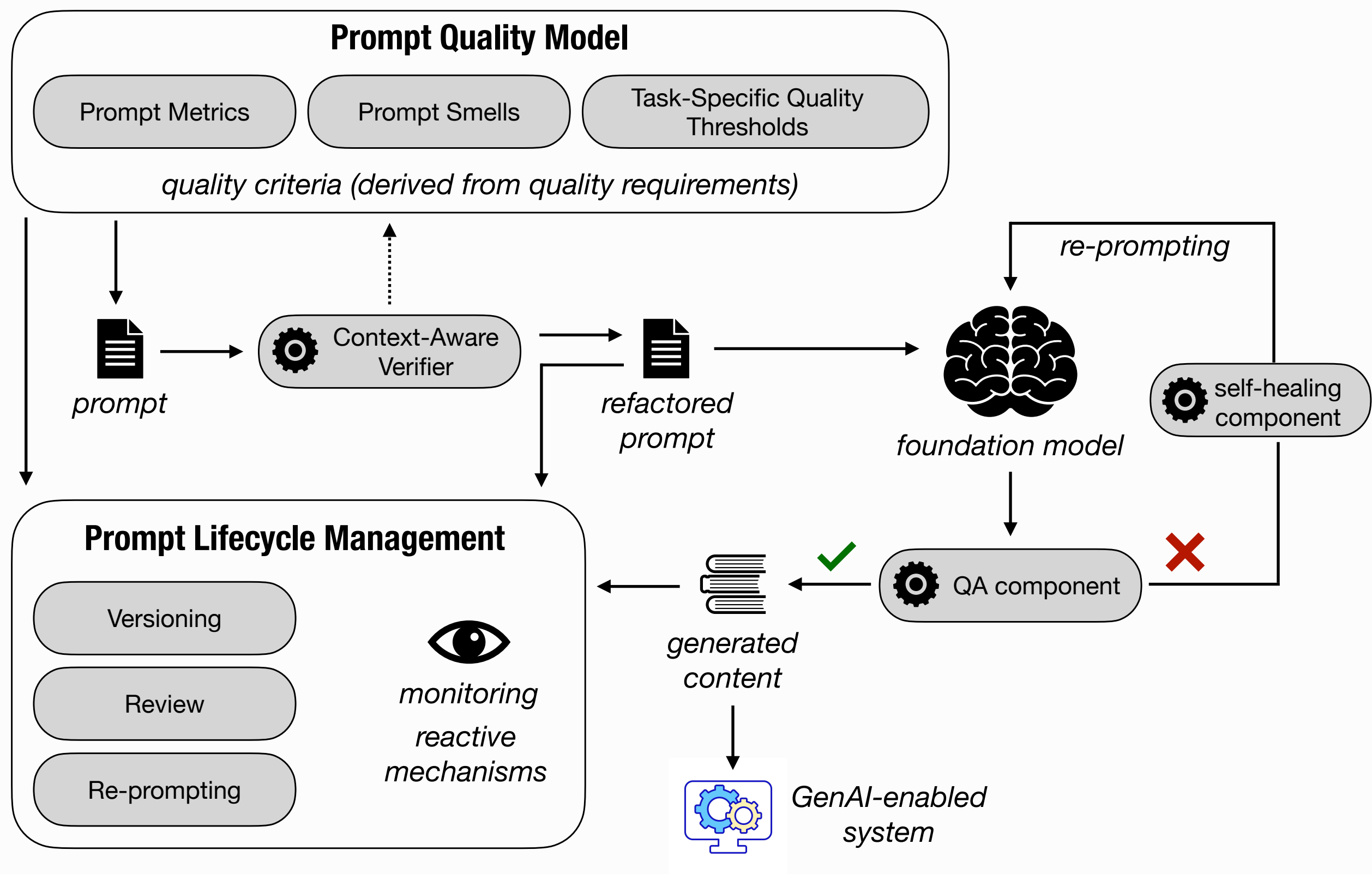
No concept of prompt regression testing, robustness testing, or metamorphic testing specifically tailored to prompts.

Automatic or semi-automatic prompt refactoring

Unlike source code, there are no tools that suggest prompt refactoring operations based on detected smells or degraded metrics.

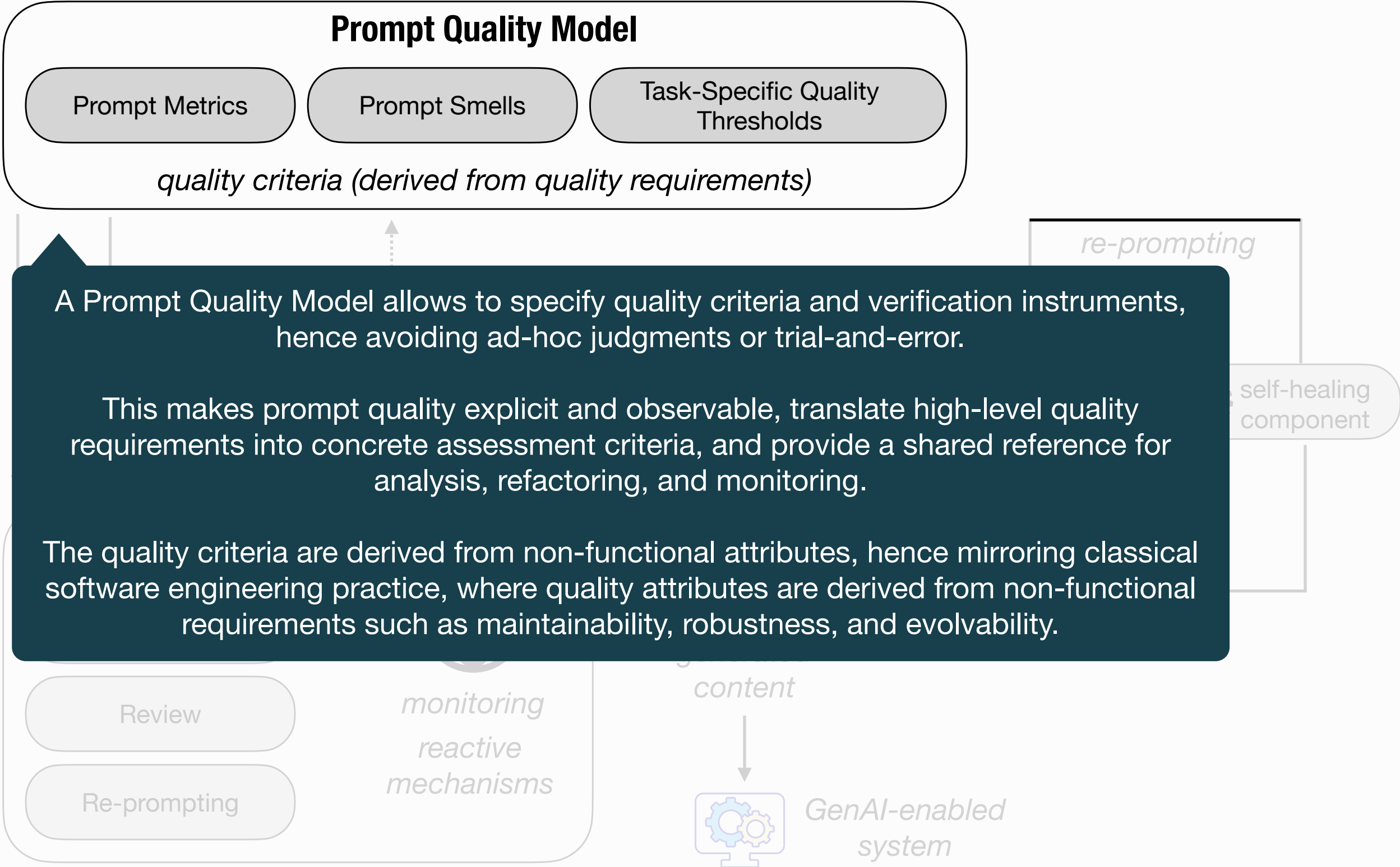
Software Engineering for Artificial Intelligence: Technical Debt in the GenAI Era

Technical Debt in the GenAI Era: Prompt Quality Assurance, The Path Forward



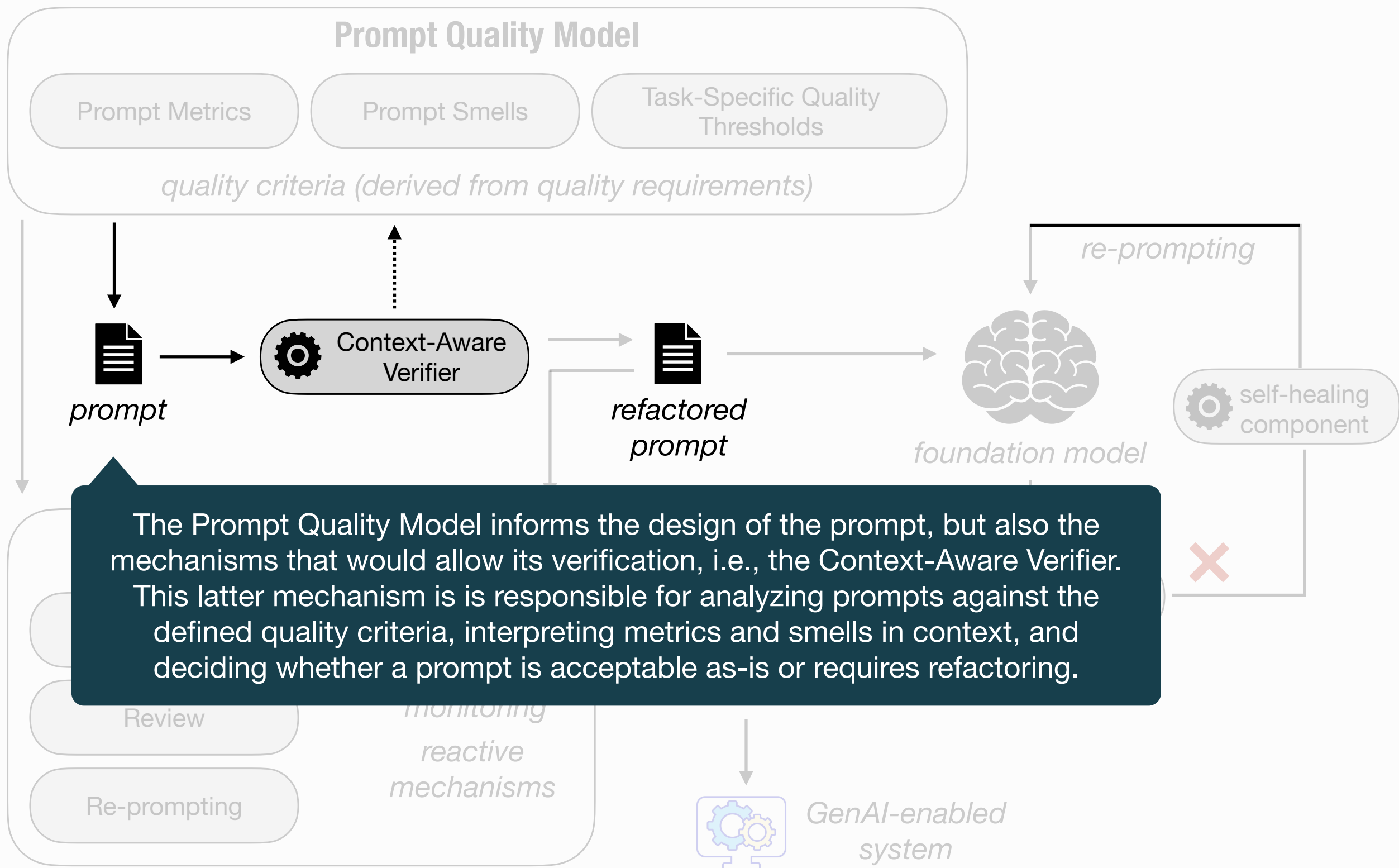
Software Engineering for Artificial Intelligence: Technical Debt in the GenAI Era

Technical Debt in the GenAI Era: Prompt Quality Assurance, The Path Forward



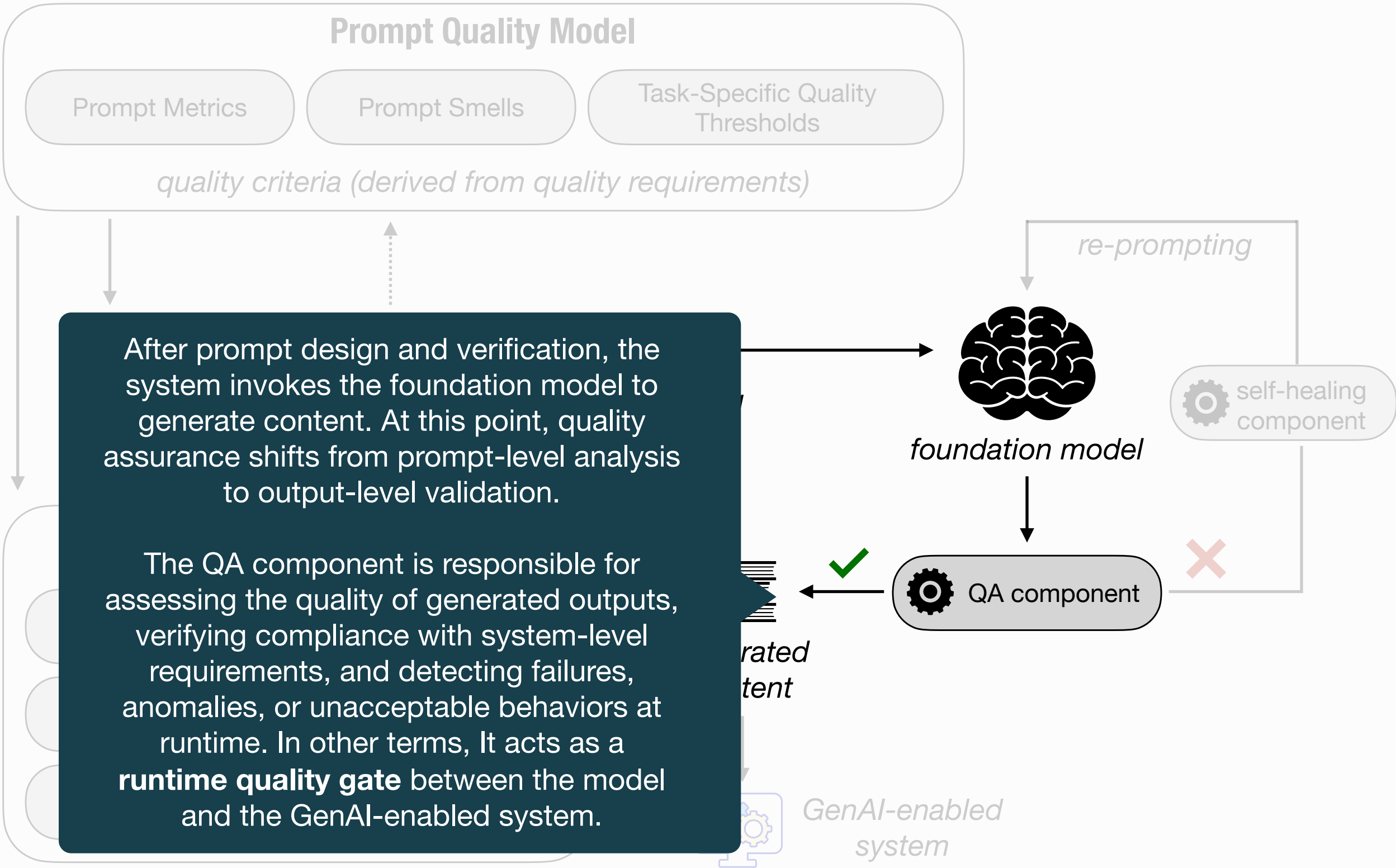
Software Engineering for Artificial Intelligence: Technical Debt in the GenAI Era

Technical Debt in the GenAI Era: Prompt Quality Assurance, The Path Forward



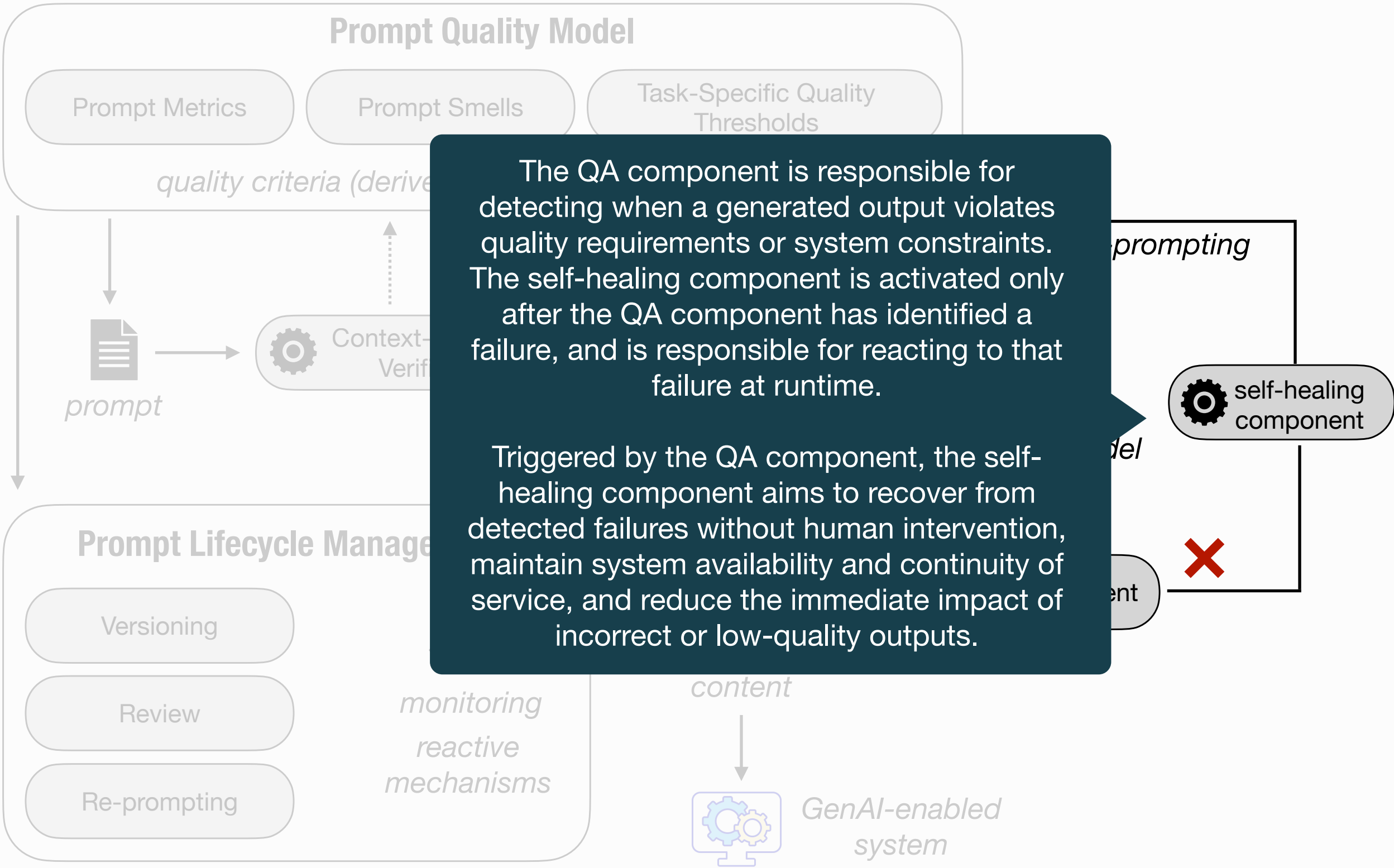
Software Engineering for Artificial Intelligence: Technical Debt in the GenAI Era

Technical Debt in the GenAI Era: Prompt Quality Assurance, The Path Forward



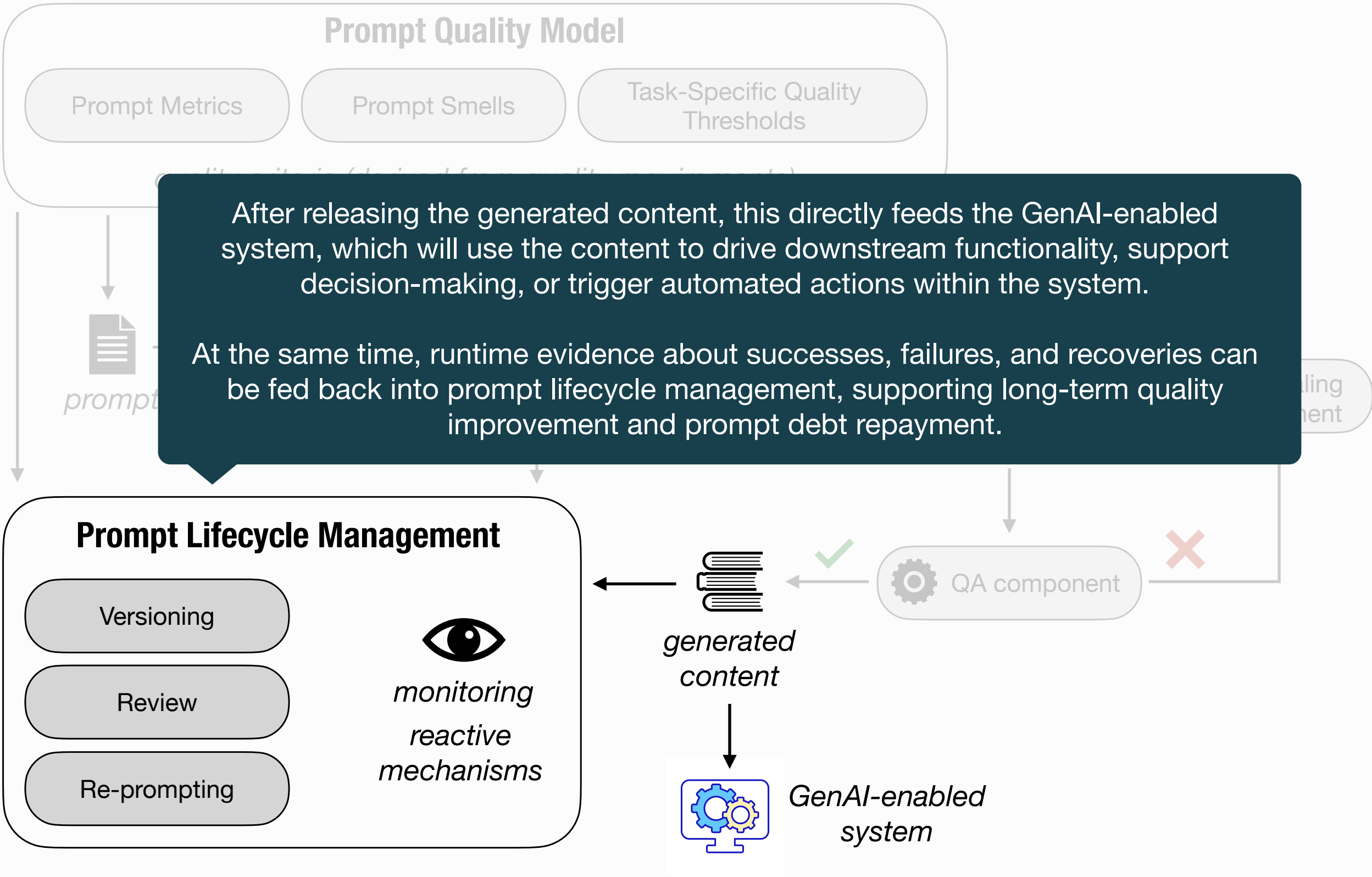
Software Engineering for Artificial Intelligence: Technical Debt in the GenAI Era

Technical Debt in the GenAI Era: Prompt Quality Assurance, The Path Forward



Software Engineering for Artificial Intelligence: Technical Debt in the GenAI Era

Technical Debt in the GenAI Era: Prompt Quality Assurance, The Path Forward



Software Engineering for Artificial Intelligence: Technical Debt in the GenAI Era

Technical Debt in the GenAI Era: What about AI Agents?

So far, we have discussed prompts as artifacts and how their quality, smells, metrics, and technical debt can be addressed using software engineering methods.

In single-prompt systems, prompt defects affect one interaction. In agentic systems, prompt defects propagate across steps, influence planning and memory, and affect downstream decisions. In other terms, prompt-related concerns may pass from being local inconsistencies to global inefficiencies.

Working Hypothesis. Prompt debt becomes architectural debt in AI Agents.

The QA vision you just saw generalizes naturally to AI agents:

Prompt Quality Model

This becomes a shared quality model across all agent prompts, ensuring consistency.

Context-Aware Verifier

Must reason not only about single prompts, but about prompt roles within the agent (planner, executor, reflector).

QA Component

Evaluates not just outputs, but agent behavior, such as plan correctness and tool usage.

Self-Healing Component

Extends from re-prompting to re-planning, re-delegation, or memory correction.

Prompt Lifecycle Management

Evolves into agent policy and prompt-set evolution, not single or isolated prompt updates.

Software Engineering for Artificial Intelligence: Technical Debt in the GenAI Era

Technical Debt in the GenAI Era: Additional Challenges of AI Agents Quality Assurance

However, there are a few, non-trivial additional challenges when dealing with AI Agents and that are due to the interactions that may happen between prompts.

First and foremost, multiple prompts interact and constrain each other: as a consequence, **quality issues can emerge from their interaction**, even when each prompt appears acceptable in isolation.

In addition, observability becomes harder, because **failures are delayed or indirect**. Indeed, errors introduced by one prompt may only surface several steps later, making it difficult to trace failures back to their original cause.

Debt accumulates faster due to iterative execution. Because agents repeatedly execute the same prompts in loops, small design shortcuts are amplified over time, accelerating the accumulation of technical debt.

Key takeaway message

If prompts are executable specifications, then AI agents are architectures built out of those specifications. At the current state of the art, we still lack a holistic view of technical debt in agentic AI systems.

This means that, from a pragmatic point of view, **most current agentic AI systems cannot yet be considered reliably engineered or systematically secure.**

Software Engineering for Artificial Intelligence: Technical Debt in the GenAI Era

Technical Debt in the GenAI Era: Some Open Questions (or Food for Thoughts)...

What does it mean to test an AI agent whose behavior emerges from multiple interacting prompts? How do we define test cases, test adequacy, and regression when the same prompt sequence may lead to different behaviors?

How should prompt review be conducted in agentic systems? Is reviewing individual prompts sufficient, or do we need review practices that consider prompt interactions and agent workflows?

Can QA for agentic AI remain output-centric, or must it shift toward behavior- and policy-centric assurance? What QA artifacts should be produced, tracked, and evolved over time?

How should testing, verification, review, and refactoring be integrated into a coherent QA process for AI agents? What is missing to turn isolated techniques into a systematic engineering practice?

How can regulation-aware guardrails be designed to address the risk of prompt interactions introducing regressions at the agent level? How can self-healing components deal with the complexity of addressing issues at agent-level?

Software Engineering for Artificial Intelligence: Verification and Validation

Before leaving... What Should You Remember?

Software Engineering for Artificial Intelligence: Technical Debt in the GenAI Era

Technical Debt in the GenAI Era: Prologue

In the previous lectures, we explored multiple aspects connected to the quality assurance of AI-enabled systems, finding that AI systems already accumulate data debt, model debt, or even system-level debt like Pipeline Jungle and similar.

Furthermore, we explored the concept of Agentic AI as a composition of software systems, not just models.

This lecture combines the two, elaborating how technical debt may emerge in GenAI and GenAI Agents. NB: Most of the arguments in the lecture are emerging, so there is no ground truth here, just reasonable consideration!

Working Hypothesis. In GenAI systems, prompts become *executable specifications*. In other terms, prompts are not documentation, but *code-like artifacts*.

The rationale behind this working hypothesis is that prompts:

1

Act as *architectural connectors* between models, tools, and goals;

2

Define system-level behavior *without being compiled or statically verified*;

3

Replace traditional control structures with *natural-language orchestration logic*;

4

Are embedded in agentic loops, where small changes can have non-local effects.

Software Engineering for Artificial Intelligence: Technical Debt in the GenAI Era

Technical Debt in the GenAI Era: Prompt Debt

Prompt Debt. It arises when prompt design decisions are deliberately optimized for short-term effectiveness or delivery speed, at the expense of long-term qualities such as maintainability, robustness, evolvability, and architectural clarity.

So, the system may possibly behave correctly today, but its design knowledge lives in the prompt, not in the architecture.

An architecturally sound solution would have been, instead:

Pre-processor

This normalizes incoming tickets and extracts structured signals, e.g., metadata and customer, context before the data is passed to the LLM.

LLM classifier

This is responsible for classifying the ticket and producing an output that includes the predicted category, an urgency score, and supporting evidence.

Validator

This checks that the LLM output conforms to a predefined schema, satisfies content constraints, and meets consistency and confidence requirements.

Policy engine

This applies explicit business rules to decide whether the ticket should be escalated, independently of how the classification was produced.

Action orchestrator

This executes system actions based on policy, enforcing permissions, safety constraints, and operational limits on any external tools or services.

Observability layer

This continuously monitors system behavior by collecting logs and traces, tracking model drift, and supporting evaluation of the agent's decisions.

Software Engineering for Artificial Intelligence: Technical Debt in the GenAI Era

Technical Debt in the GenAI Era: Prompt Debt

Prompt Debt. It arises when prompt design decisions are deliberately optimized for short-term effectiveness or delivery speed, at the expense of long-term qualities such as maintainability, robustness, evolvability, and architectural clarity.

Prompt Smell. It is a recurring design weakness in prompts that may not cause immediate failures, but that undermine the long-term quality of LLM-enabled systems.

As in the case of traditional code smells, they are not bugs (namely, they do cause failures), but are indicators of potential technical debt.

At the time of this lecture, we still know only a few things about prompt smells - we are indeed in the moment where we can just classify few types based on syntax or semantic.

Perhaps more importantly, most of the existing prompt smells refer to text-to-text GenAI models/systems, hence missing other types of GenAI (e.g., text-to-image, text-to-video, image-to-video). The missing ones are very likely the most problematic!

Even more importantly, prompt debt has not yet been formally defined - the definition and example discussed so far are my own interpretation, inspired by classical technical debt theory.

In this lecture, we treat prompt smells as early warning signals of prompt debt.

Software Engineering for Artificial Intelligence: Technical Debt in the GenAI Era

Technical Debt in the GenAI Era: Prompt Debt

Prompt Smell. It is a recurring design weakness in prompts that may not cause immediate failures, but that undermine the long-term quality of LLM-enabled systems.

As some of you may have heard already, smells are typically diagnosed - and potentially automatically detected - through static analysis tools (or through advanced mechanisms, like machine learning or even Large Language Models).

Independently from the mechanism used for detection, a key enabler for **making smells observable** is represented by the ability to **associate artifacts with measurable indicators**.

Prompt Metric. A prompt metric is a quantitative indicator that captures a specific, observable property of a prompt as a software artifact.

More specifically, prompt metrics are used to approximate different dimensions of prompt quality, e.g., prompt length. Also, to support the identification and assessment of prompt smells, e.g., by defining thresholds over length to identify excessively long prompts. And, thus, to enable systematic analysis, comparison, and monitoring of prompts over time.

NB: A prompt metric does **not** directly measure correctness or usefulness.

Software Engineering for Artificial Intelligence: Technical Debt in the GenAI Era

Technical Debt in the GenAI Era: Prompt Quality Assurance, The Path Forward

```
graph TD
    subgraph PQM [Prompt Quality Model]
        PM[Prompt Metrics]
        PS[Prompt Smells]
        TSQT[Task-Specific Quality Thresholds]
        QC[quality criteria derived from quality requirements]
    end
    subgraph PLM [Prompt Lifecycle Management]
        V[Versioning]
        R[Review]
        RP[Re-prompting]
        MR[monitoring reactive mechanisms]
    end
    P[prompt] --> CAV[Context-Aware Verifier]
    CAV --> PQM
    CAV --> RP1[refactored prompt]
    RP1 --> FM[foundation model]
    FM --> QA[QA component]
    QA --> PLM
    QA --> SC[self-healing component]
    SC --> RP2[re-prompting]
    RP2 --> CAV
    QA --> GC[generated content]
    GC --> GES[GenAI-enabled system]
```

Software Engineering for Artificial Intelligence: Technical Debt in the GenAI Era

Technical Debt in the GenAI Era: What about AI Agents?

So far, we have discussed prompts as artifacts and how their quality, smells, metrics, and technical debt can be addressed using software engineering methods.

In single-prompt systems, prompt defects affect one interaction. In agentic systems, prompt defects propagate across steps, influence planning and memory, and affect downstream decisions. In other terms, prompt-related concerns may pass from being local inconsistencies to global inefficiencies.

Working Hypothesis. Prompt debt becomes architectural debt in AI Agents.

The QA vision you just saw generalizes naturally to AI agents:

Prompt Quality Model

This becomes a shared quality model across all agent prompts, ensuring consistency.

Context-Aware Verifier

Must reason not only about single prompts, but about prompt roles within the agent (planner, executor, reflector).

QA Component

Evaluates not just outputs, but agent behavior, such as plan correctness and tool usage.

Self-Healing Component

Extends from re-prompting to re-planning, re-delegation, or memory correction.

Prompt Lifecycle Management

Evolves into agent policy and prompt-set evolution, not single or isolated prompt updates.



Software Engineering for Artificial Intelligence

Technical Debt in the GenAI Era

